

Fundamental Programming Concepts and Data Structures

JAHEZIA

**College of Science
Northern Border University**



Primitive Data types, standard conditional and iterative control structures and functions

Primitive data types, basic conditional and loop structures, and functions are core programming concepts present in most modern languages. They serve as essential building blocks for storing data, directing a program’s flow, and organizing code into reusable units.

1. Primitive Data Types

Common primitive types are in below table:

TYPE	DESCRIPTION	EXAMPLE VALUES
INTEGER (INT)	WHOLE NUMBERS	1, -20, 500
FLOATING-POINT (FLOAT/DOUBLE)	DECIMAL NUMBERS	3.14, -0.001
CHARACTER (CHAR)	SINGLE CHARACTER	'A', 'Z', '1'
BOOLEAN (BOOL)	TRUE/FALSE VALUES	TRUE, FALSE
BYTE/SHORT/LONG	VARIOUS INTEGER SIZES	8-BIT, 16-BIT, 64-BIT
STRING*	A SEQUENCE OF CHARACTERS (PRIMITIVE IN SOME LANGUAGES, LIKE PYTHON)	"HELLO"

2. Standard Control Structures: Control structures determine the order in which instructions are executed in a program there are two types of Standard Control structures.

a. Conditional Control Structures

b. Iterative Control Structures (Loops)

a. Conditional Control Structures: These structures choose which code block to run based on a condition.

if, else if, else: Runs the first block whose condition is true; the final else runs if none are.

switch/case: Selects one of many blocks based on a single value, offering a cleaner alternative to multiple else-if checks.

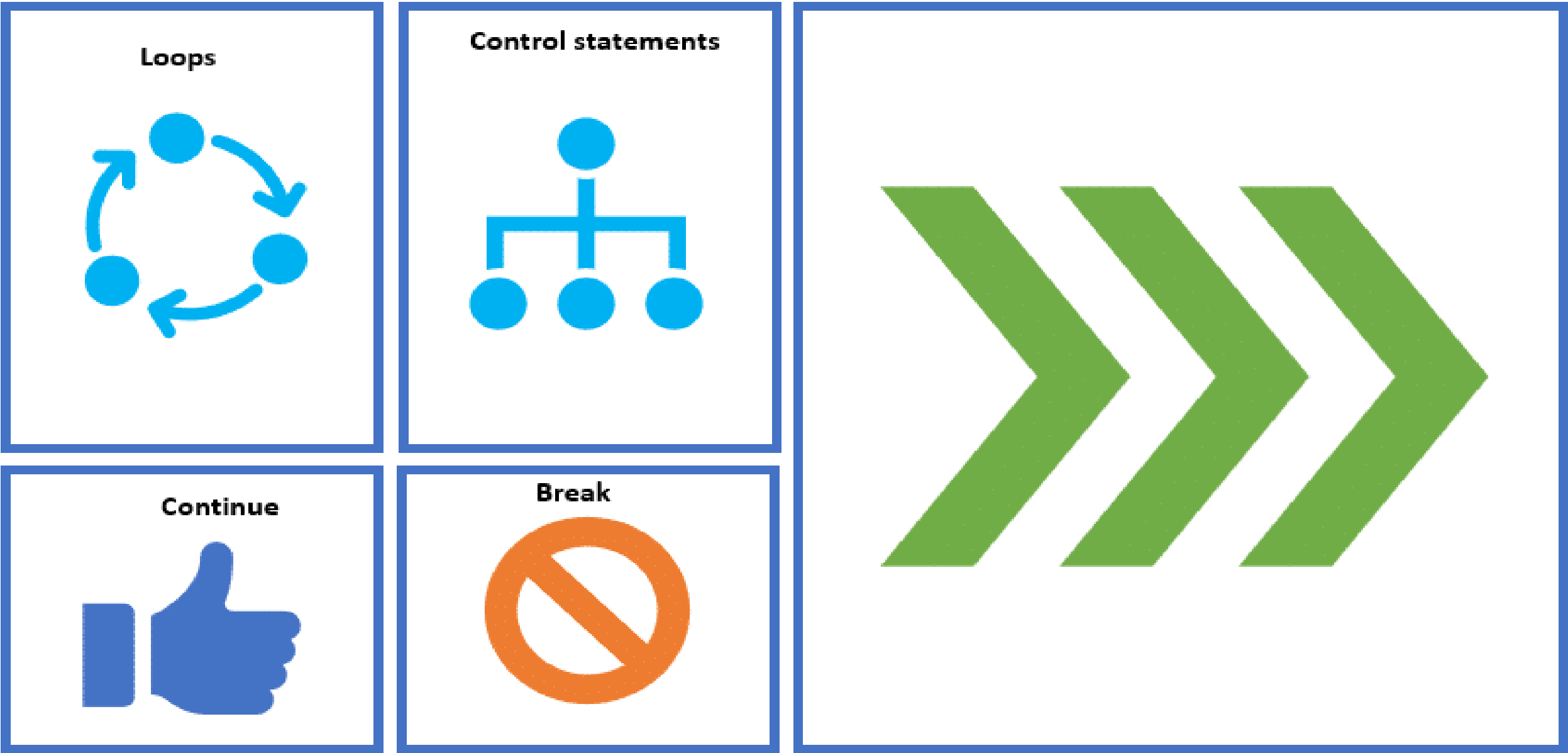
• IF STATEMENT : <pre>IF (X > 0) { PRINTF("POSITIVE"); }</pre>	ELSE IF / ELIF <pre>IF (SCORE >= 90) { GRADE = 'A'; } ELSE IF (SCORE >= 80) { GRADE = 'B'; } ELSE { GRADE = 'C'; }</pre>
• IF-ELSE: <pre>INT X = /* YOUR VALUE */; IF (X % 2 == 0) { SYSTEM.OUT.PRINTLN("EVEN"); } ELSE { SYSTEM.OUT.PRINTLN("ODD"); }</pre>	SWITCH / MATCH <pre>SWITCH(DAY) { CASE 1: SYSTEM.OUT.PRINTLN("MONDAY"); BREAK; CASE 2: SYSTEM.OUT.PRINTLN("TUESDAY"); BREAK; DEFAULT: SYSTEM.OUT.PRINTLN("UNKNOWN"); };</pre>

b. Iterative Control Structures (Loops)

These structures repeat a block of code until a specific condition is met or for a fixed number of times.

- **for loop:** Used when the number of iterations is known; includes initialization, a condition, and an update step.
- **while loop:** Repeats code while a condition is true, checking the condition before each iteration.
- **do-while loop:** Like a while loop but guarantees the code runs at least once before checking the condition.

FOR LOOP <pre>FOR (INT I = 0; I < 5; I++) { SYSTEM.OUT.PRINT(I); }</pre>	FOR-EACH LOOP (COLLECTION ITERATION) <pre>INT[] ITEMS = {1, 2, 3}; FOR (INT ITEM : ITEMS) { SYSTEM.OUT.PRINTLN(ITEM); }</pre>
WHILE LOOP <pre>INT COUNT = 0; WHILE (COUNT < 5) { SYSTEM.OUT.PRINTLN(COUNT); COUNT++; }</pre>	DO-WHILE LOOP (RUNS AT LEAST ONCE) <pre>DO { N--; } WHILE (N > 0);</pre>



Functions (Methods/Procedures)

Functions are named, reusable code blocks designed to perform a specific task. They are fundamental to structured programming.

Purpose:

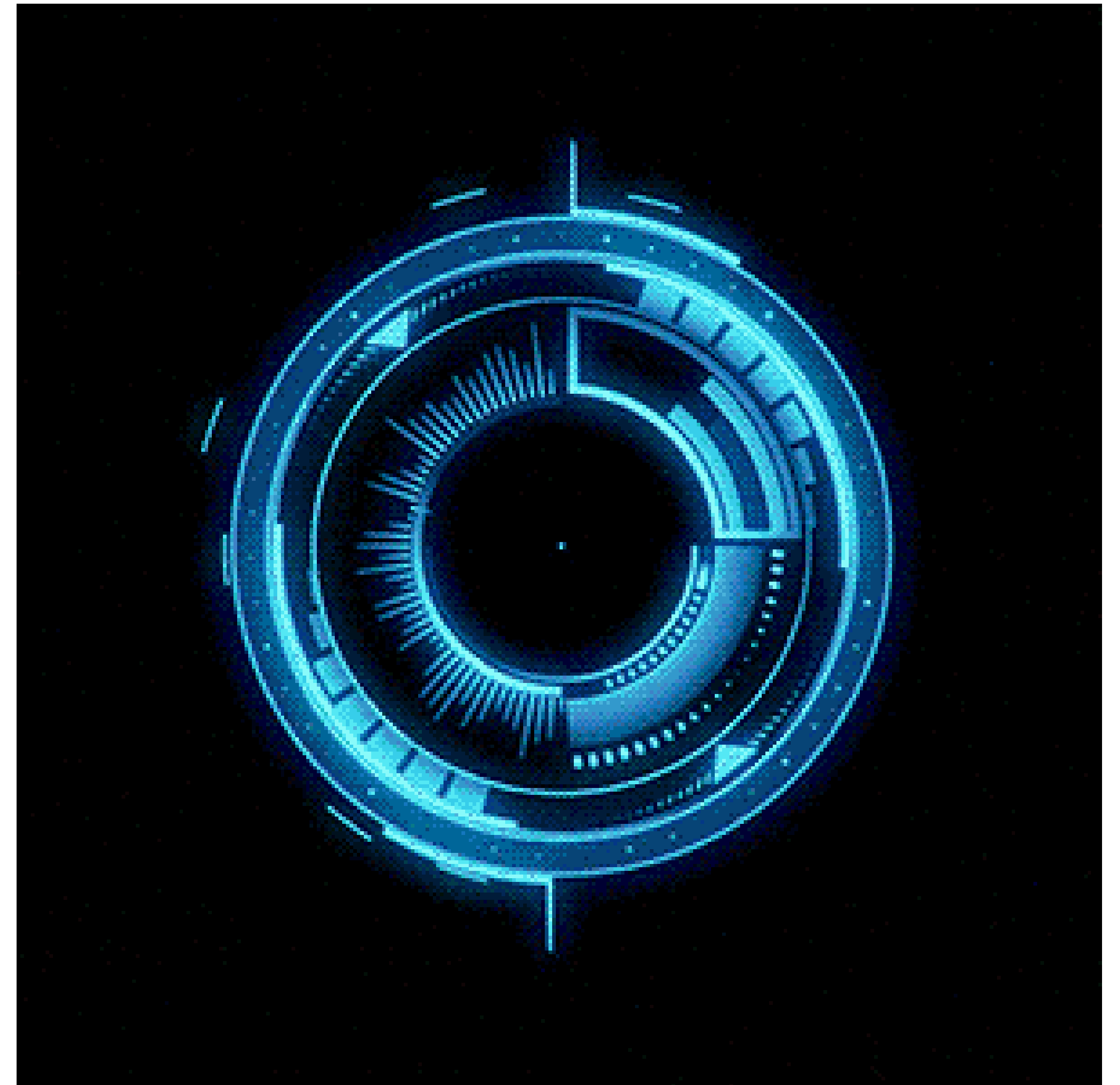
They break large programs into smaller parts, improve clarity and organization, and reduce duplication by following the DRY (Don't Repeat Yourself) principle.

Components:

- > Name: Identifies the function when calling it.
- > Parameters: Inputs the function receives.
- > Body: The code executed when the function runs.
- > Return Value: The result the function sends back to the caller (if any).

Example :

```
public static int add(int a, int b) {  
    return a + b;  
}
```



Usage of the following fundamental programming constructs: basic computation, simple I/O, file I/O, standard conditional and iterative structures, the definition of functions, and parameter passing.

The fundamental programming constructs are the building blocks used to create software. Their usage is as follows:

- 1. **Basic Computation:** Performs arithmetic, logical, and relational operations on data, used for calculations, comparisons, and decision-making.
- 2. **Simple I/O:** Allows programs to interact with users by reading input and displaying output, facilitating communication with the program.
- 3. **File I/O:** Enables reading from and writing to files, allowing persistent storage and retrieval of program data.
- 4. **Standard Conditional Structures:** Control the flow of execution based on conditions, using constructs like if, if-else, and switch.
- 5. **Iterative Structures:** Repeat a block of code until a condition is met, using loops such as for, while, and do-while.
- 6. **Functions (Methods/Subroutines):** Encapsulate reusable blocks of code to perform specific tasks, which may accept parameters and return values.
- 7. **Parameter Passing:** Mechanism to provide data to functions or methods; primitives are passed by value, while objects and arrays are passed by reference.

CONSTRUCT	PURPOSE	EXAMPLE
BASIC COMPUTATION	PERFORM ARITHMETIC & LOGIC	A+B, X>Y
	COMMUNICATE WITH USER	SCANNER / SYSTEM.OUT
	READ/WRITE FILES	FILEREADER / FILEWRITER
CONDITIONAL	DECISION-MAKING	IF, SWITCH
ITERATIVE	REPETITION	FOR, WHILE, DO-WHILE
FUNCTIONS	REUSABLE CODE BLOCKS	INT ADD(A,B)
PARAMETER PASSING	TRANSFER DATA TO METHODS	INCREMENT(N)

Expression Optimizer :

- **Function:** A phase in language processing (compiler or interpreter design) focused on improving the quality and efficiency of the code, typically in terms of speed, memory usage, or storage size.
- **Process:** Optimizers apply various transformations (e.g., constant folding, dead code elimination, loop optimization) to make the code run more efficiently without changing its overall function.

- **Characteristics:**

It can operate on an intermediate representation of the code before final execution or code generation.

Optimizations can be "local" (within a basic block of code) or "global" (across entire functions or modules).

Expression optimizers specifically target individual expressions, for instance, replacing $3.14 > 2$ and $3.15 < 1$ with its boolean result at compile time if the values are constant.

Documentation Generator :

- **Function:** A software tool that automatically generates structured, often hyperlinked, human-readable documentation from source code and embedded comments (like Javadoc or Doxygen).
- **Process:** It parses the source code to extract structural information (classes, functions, parameters, data types) and specially formatted comments. It then formats this information into various output formats (e.g., HTML, PDF, Markdown).

- **Characteristics:**

Automation: Reduces the manual effort required to keep documentation synchronized with code changes.

Standardization: Enforces a consistent format and style for documentation across a project.

Integration: Becomes a standard part of the software development lifecycle and Continuous Integration/Continuous Deployment (CI/CD) pipelines.

Concept of recursion

Recursion is a programming technique in which a function calls itself directly or indirectly to solve a problem. Each recursive call should simplify the problem, and there must be a base case to terminate the recursion; otherwise, it leads to infinite calls.

- Key idea: Break a problem into smaller instances of the same problem.
- Base case: Condition under which the recursion stops.
- Recursive case: The part where the function calls itself.

Recursion Concept

- **Base Case:** A condition that ends the recursion, giving a direct solution to the simplest form of the problem. Without it, the function would loop indefinitely.
- **Recursive Case:** The portion where the function calls itself with a smaller or simpler version of the original problem.

Recursion Examples:

• **Factorial:** To calculate $n!$, multiply n by the factorial of $(n-1)$.

- **Base Case:** $0! = 1$ or $1! = 1$
- **Recursive Case:** $n! = n \times (n-1)!$

• **Sum of Natural Numbers:** To find the sum of the first n natural numbers, add n to the sum of the first $(n-1)$ numbers.

- **Base Case:** $\text{sum}(1) = 1$
- **Recursive Case:** $\text{sum}(n) = n + \text{sum}(n-1)$

• **Fibonacci Sequence:** Each number equals the sum of the two preceding numbers.

- **Base Case:** The first two numbers (e.g., 0 and 1) are given directly
- **Recursive Case:** $\text{fib}(n) = \text{fib}(n-1) + \text{fib}(n-2)$

• **Directory Traversal:** To list all files in a folder and its subfolders, first list files in the current folder, then recursively list files in each subfolder.

- **Base Case:** A folder with no subfolders
- **Recursive Case:** A folder containing subfolders, prompting recursive calls for each one

Low-level runtime representation of core language constructs.

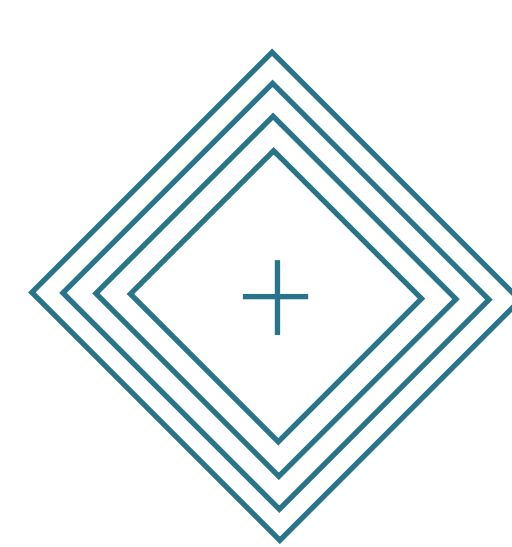
Low-level runtime representation of core language constructs refers to how the fundamental elements of a programming language are represented and managed in memory during program execution. It's the bridge between high-level code and the machine-level operations that actually run.

Examples:

- **Variables** → memory addresses
- **Functions** → stack frames, return addresses
- **Objects/structs** → contiguous or pointer-linked memory blocks
- **Arrays** → fixed-size blocks indexed by offset
- **Control flow** → instruction pointers, jump tables

Key points:

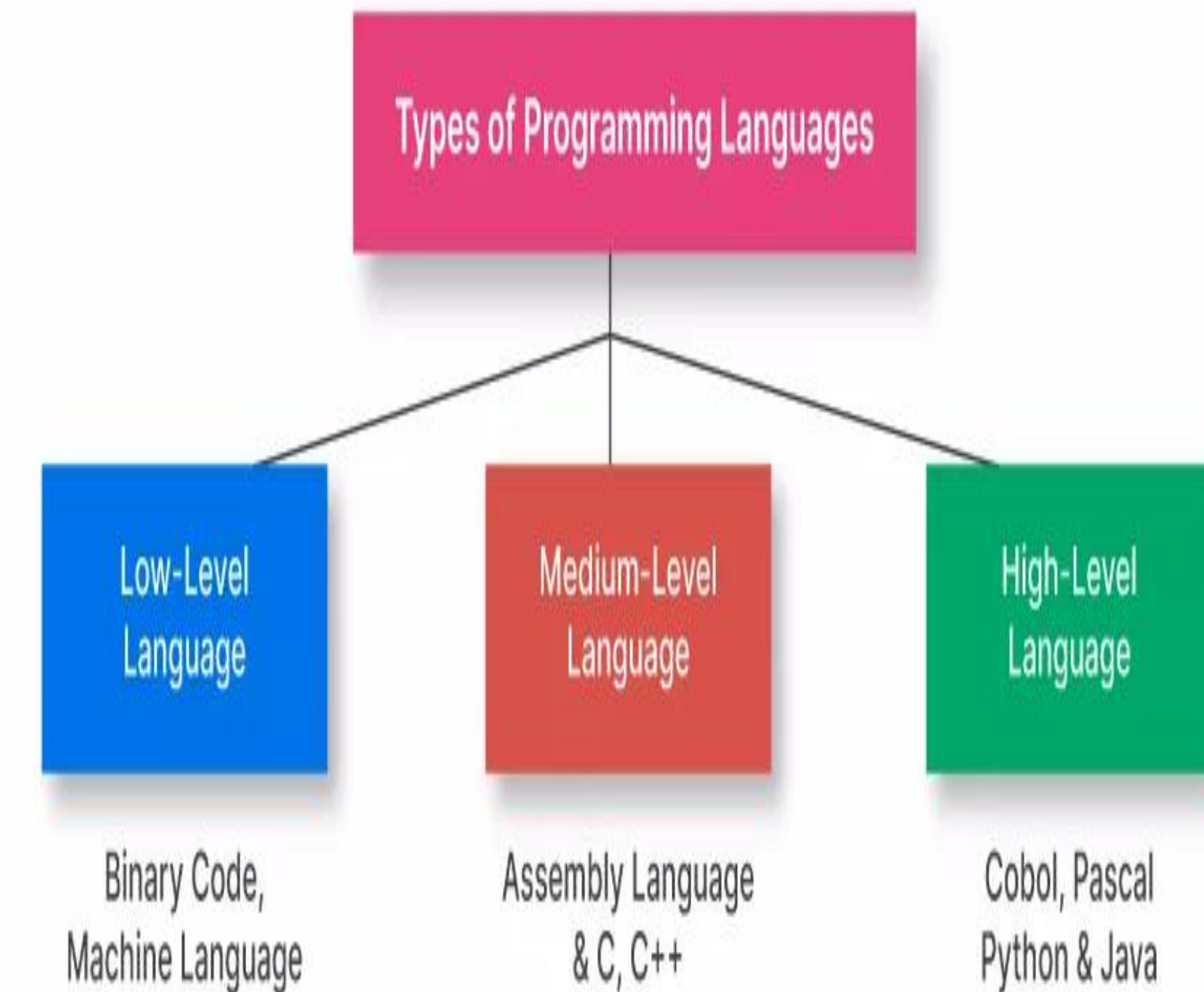
- **Variables and Data** – stored in memory (stack, heap, registers) with specific formats depending on type.
- **Control Structures** – loops, conditionals, and function calls are represented as sequences of instructions or control flow constructs at runtime.
- **Functions/Procedures** – activation records or call frames track local variables, parameters, and return addresses.
- **Expressions and Operations** – evaluated according to runtime rules; often compiled to machine instructions or intermediate code.
-

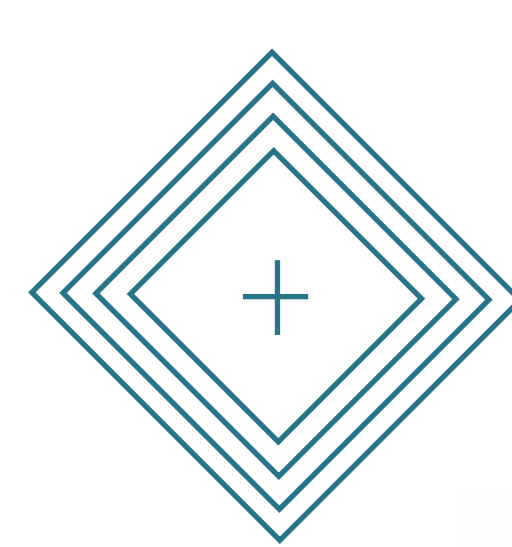


Difference Between High-level And Low-level Programming Languages

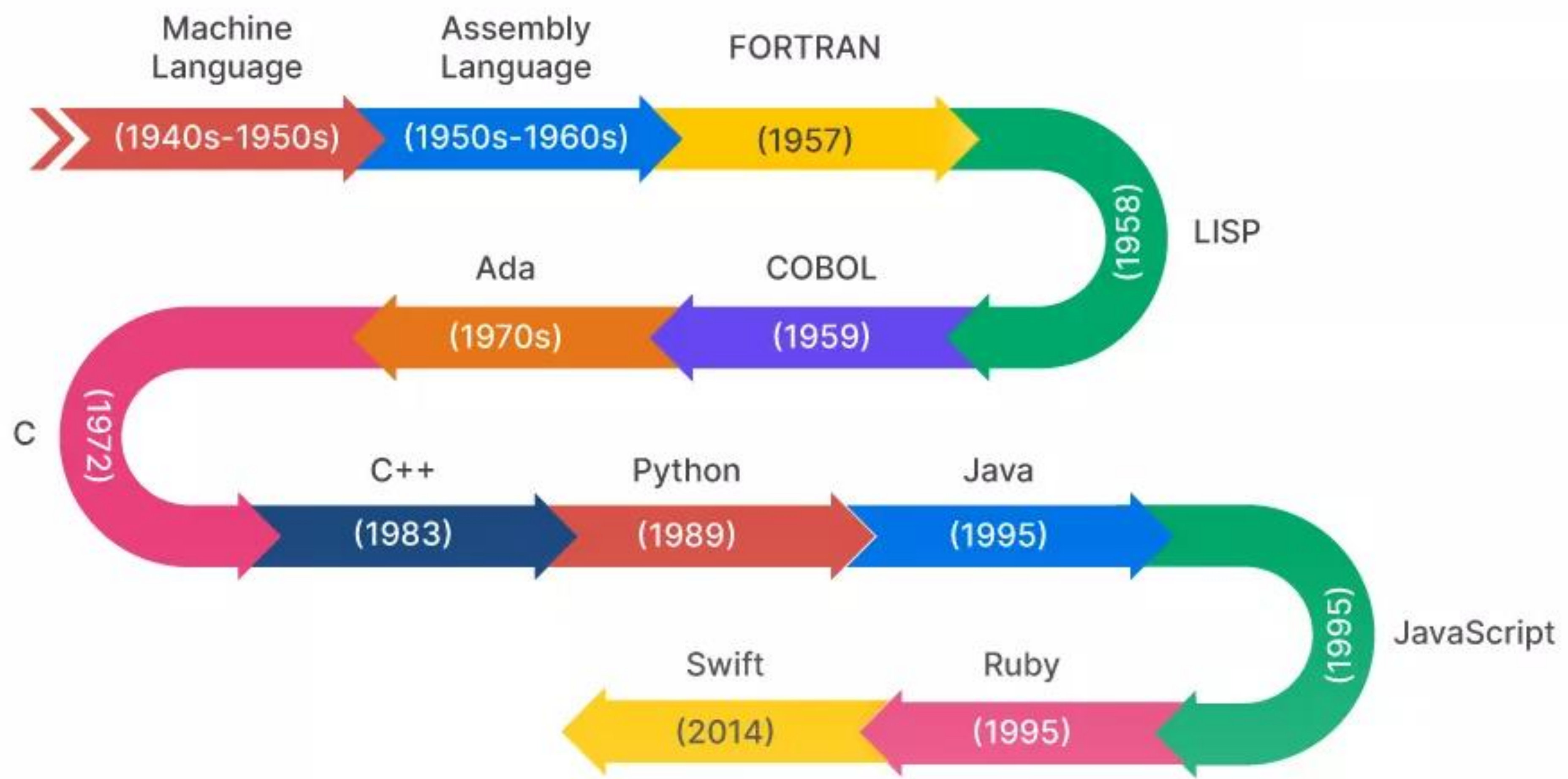
Low-level Languages	High-level Languages
• Easy to understand for machine, but less human-readable • Not Portable • Higher execution speed • Uses binary code and mnemonics • Examples: Assembly language, Machine language	• Human-readable and easier to understand and use. • Portable • Comparatively lower execution speed • Uses syntax and semantics • Examples: Java language, Python, C++

Types of Programming Language





The Evolution Of High-level And Low-level Languages



Use the following data structures: arrays, records, strings, linked lists, stacks, queues, sets, and maps.

1. Arrays : An array is a data structure that stores multiple elements of the same type under one variable name. Each element is accessed by an index indicating its position, and the elements are kept in contiguous memory for fast access.

Common uses:

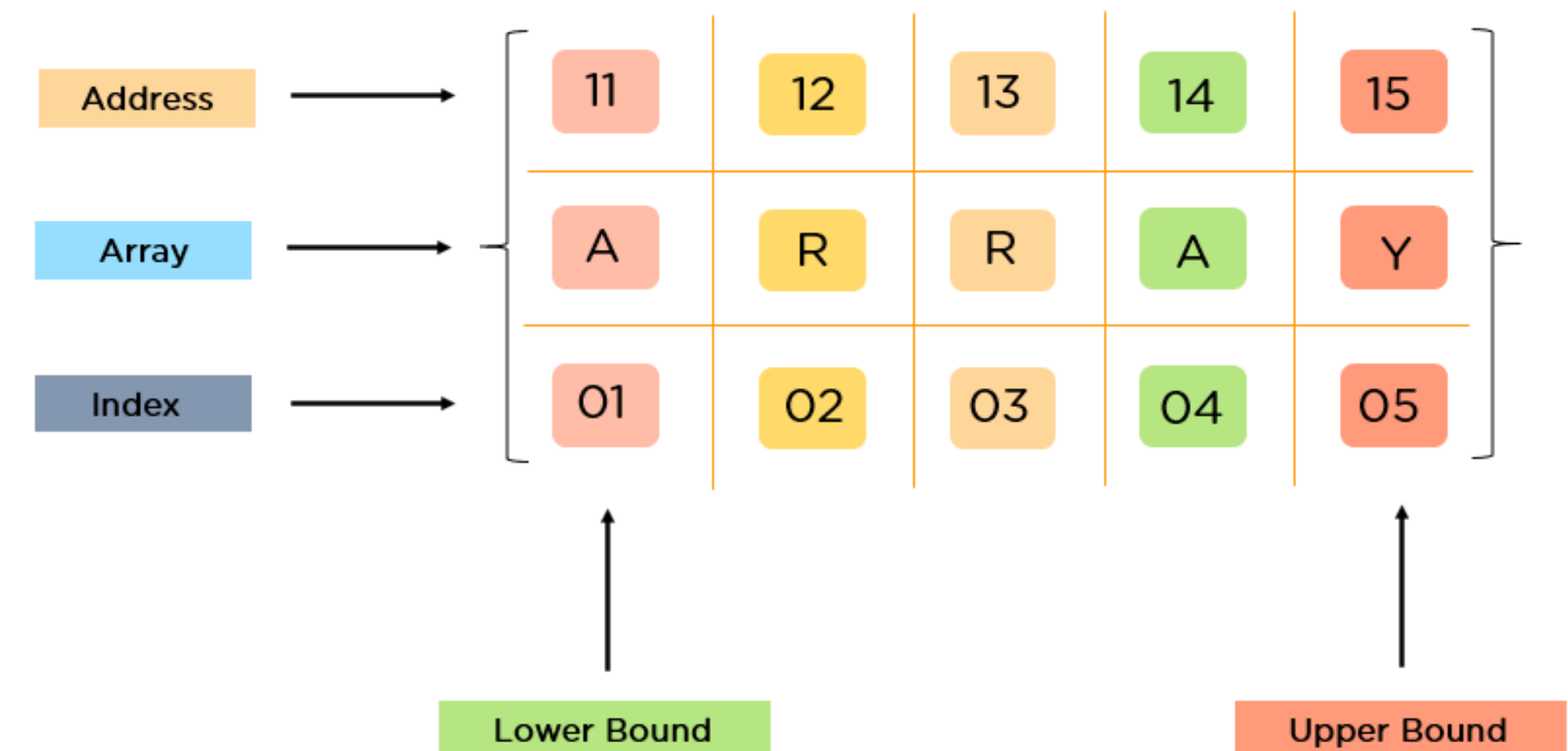
Storing data: Storing large amounts of similar data, like a list of customer names or product prices.

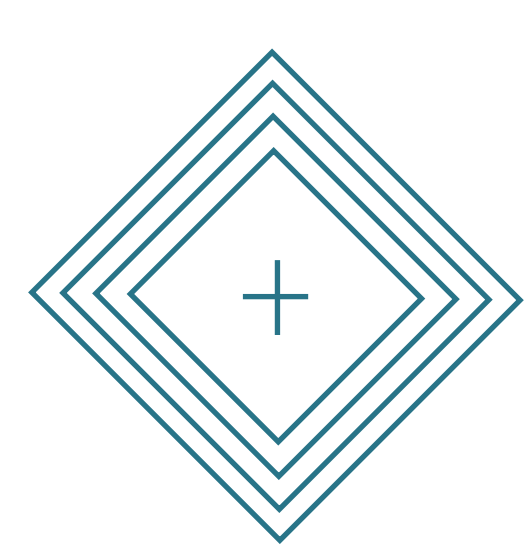
Implementing other data structures: Arrays are often used as the underlying structure for other data types like stacks, queues, and heaps.

Performing operations: Easily loop through and perform operations on each element, no matter how large the array is.

Key characteristics :

- **Collection of elements:** Stores multiple related values.
- **Single name:** All elements share one variable name for easy management.
- **Indexed:** Each element is accessed by a numeric index (often starting at 0).
- **Contiguous memory:** Elements are stored consecutively for fast access.
- **Uniform data type:** Typically holds elements of the same type (with exceptions in languages like Python).
- **Multiple dimensions:** Can be one-dimensional or multi-dimensional (e.g., tables or grids).





Arrays Example in Java

```
public class ArrayExample {  
    public static void main(String[] args) {  
        int[] numbers = {10, 20, 30, 40, 50};  
  
        System.out.println("First element: " + numbers[0]);  
        System.out.println("Third element: " + numbers[2]);  
        System.out.println("All elements:");  
  
        for (int i = 0; i < numbers.length; i++) {  
            System.out.println(numbers[i]);  
        }  
    }  
}
```

Program Output :

First element: 10
Third element: 30
All elements:
10
20
30
40
50



2. Strings

A **string** is a sequence of characters. It is commonly used for **text processing, file parsing, messages, commands**, and any situation where characters need to be stored or manipulated.

```
public class StringExample {
    public static void main(String[] args) {
        String message = "Hello World";

        System.out.println("Message: " + message);
        System.out.println("First character: " +
message.charAt(0));
        System.out.println("Length: " + message.length());
    }
}
```

Output :

```
Message: Hello World
First character: H
Length: 11
```

3. Records(Structs):

A record (also called a struct, object, or tuple depending on the language) is a data structure that groups together related fields that may be of different data types. Use a record when you need to represent one **entity or thing** with multiple attributes. Examples: a student (name, age, grade), a car (model, year, price), a point (x, y).

```
class Student {
    String name;
    int age;
    double grade;
}

public class RecordExample {
    public static void main(String[] args) {
        Student s = new Student();
        s.name = "Alice";
        s.age = 20;
        s.grade = 88.5;
        System.out.println("Name: " + s.name);
        System.out.println("Age: " + s.age);
        System.out.println("Grade: " + s.grade);
    }
}
```

Output:

```
Name: Alice
Age: 20
Grade: 88.5
```

4. Linked List .

Linked List: A **linked list** is a data structure where elements (called **nodes**) are stored **non-contiguously** in memory, and each node contains:

- **Data** – the value of the element.
- **Next** – a reference (or pointer) to the next node in the sequence.

Key Features:

- Dynamic size (can grow/shrink at runtime)
- Efficient insertion/deletion (especially at the beginning or middle)
- Sequential access (slower than arrays for random access)

When to use:

- When the number of elements changes frequently.
- When fast insertion/deletion is needed, but random access is not.

Example : singly Liked list :

```
class Node {
    int data;
    Node next;

    Node(int data) {
        this.data = data;
        this.next = null;
    }
}

public class LinkedListExample {
    public static void main(String[] args) {
        Node head = new Node(10);
        head.next = new Node(20);
        head.next.next = new Node(30);

        Node current = head;
        while(current != null) {
            System.out.println(current.data);
            current = current.next;
        }
    }
}
```

Output :

10
20
30

5. Stacks

Stack: A stack is a linear data structure that follows LIFO (Last In, First Out) principle.

- The last element added is the first one removed.
- Main operations:
 - > push() – add an element to the top
 - > pop() – remove the top element
 - > peek() / top() – view the top element without removing

When to use:

- Undo mechanisms in editors
- Expression evaluation (e.g., parentheses matching, postfix evaluation)
- Function call management (call stack in programming languages)

Example :

```
import java.util.Stack;
public class StackExample {
    public static void main(String[] args) {
        Stack<Integer> stack = new Stack<>();
        stack.push(10);
        stack.push(20);
        stack.push(30);

        System.out.println("Top element: " + stack.peek());
        stack.pop();

        System.out.println("Stack after pop:");
        for(int item : stack) {
            System.out.println(item);
        }
    }
}
```

Output :

```
Top element: 30
Stack after pop:
10
20
```

6.Queues:

A queue is a linear data structure that follows FIFO (First In, First Out) principle.

- The first element added is the first one removed.
- Main operations:
 - > enqueue() – add an element to the rear
 - > dequeue() – remove an element from the front
 - > peek() / front() – view the front element without removing

When to use:

- Task scheduling (CPU or printer queues)
- Handling requests in order (e.g., ticketing system)
- Breadth-First Search (BFS) in graphs

Example:

```
import java.util.LinkedList;
import java.util.Queue;
```

```
public class QueueExample {
    public static void main(String[] args) {
        Queue<Integer> queue = new LinkedList<>();

        queue.add(10); // enqueue
        queue.add(20);
        queue.add(30);

        System.out.println("Front element: " + queue.peek());

        queue.remove(); // dequeue

        System.out.println("Queue after dequeue:");
        for(int item : queue) {
            System.out.println(item);
        }
    }
}
```

Output:

```
Front element: 10
Queue after dequeue:
20
30
```

7.Sets

A **set** is a data structure that stores **unique elements** with **no particular order**.

- **Duplicates are not allowed.**
- Common operations:
 - > **add()** – insert an element
 - > **remove()** – delete an element
 - > **contains()** – check if an element exists
 - > **union, intersection, difference** – for combining or comparing

sets

When to use:

- To store distinct items
- Removing duplicates from a collection
- Membership checking

Example

```
import java.util.HashSet;  
import java.util.Set;
```

```
public class SetExample {  
    public static void main(String[] args) {  
        Set<String> fruits = new HashSet<>();  
  
        fruits.add("Apple");  
        fruits.add("Banana");  
        fruits.add("Cherry");  
        fruits.add("Apple"); // duplicate, ignored  
  
        System.out.println("Fruits set:");  
        for(String fruit : fruits) {  
            System.out.println(fruit);  
        }  
  
        System.out.println("Contains Banana? " +  
fruits.contains("Banana"));  
    }  
}
```

Output :

```
Fruits set:  
Apple  
Banana  
Cherry  
Contains Banana?  
true
```

Dynamic and static data structure implementations.

Data structures can be broadly categorized into static and dynamic based on their memory allocation and size flexibility.

1. Static Data Structures: These data structures have a fixed size that is determined at compile time or when they are declared. Their memory allocation cannot change during program execution.

Implementation:

Arrays: A common example where a contiguous block of memory is allocated for a fixed number of elements. Accessing elements by index is highly efficient.

Static Stacks and Queues: Often implemented using arrays with a fixed maximum capacity.

Characteristics:

- **Fixed Size:** Cannot grow or shrink during runtime.
- **Compile-time Allocation:** Memory is allocated when the program is compiled or the variable is declared.
- **Efficient Access:** Direct, index-based access to elements is generally very fast.
- **Potential for Inefficiency:** If the allocated memory is not fully utilized, it can lead to wasted space. If the data exceeds the allocated size, overflow occurs.

2. Dynamic Data Structures:

These data structures can grow or shrink in size during program execution, adapting to the changing needs of the data. Memory is allocated and deallocated at runtime.

Implementation:

- **Linked Lists:** Elements (nodes) are not stored contiguously in memory but are linked together using pointers. This allows for efficient insertion and deletion anywhere in the list.
- **Trees (e.g., Binary Search Trees, AVL Trees):** Hierarchical structures where nodes are connected via pointers, allowing for dynamic growth and efficient search, insertion, and deletion operations.
- **Hash Tables:** Often implemented using arrays of linked lists or other dynamic structures to handle collisions, allowing for dynamic resizing.
- **Dynamic Stacks and Queues:** Can be implemented using linked lists, allowing them to grow or shrink as needed.

Characteristics of Dynamic Data Structures :

- **Variable Size:** Can grow or shrink at runtime.
- **Runtime Allocation:** Memory managed dynamically (e.g., malloc/free in C, new/delete in C++).
- **Flexible:** Suitable when data size is unknown or changes frequently.
- **Complex Memory Management:** Requires careful pointer handling; slower access due to pointer traversal and risk of memory leaks.